

UNIDAD 4.

ESTRUCTURAS DE DATOS ESTÁTICAS Y STRINGS

**Programación
CFGS DAW**

Autores:
José Miguel Blázquez
Carlos Cacho
Raquel Torres
Lionel Tarazon
Fco. Javier Valero

2023/2024

Licencia



Reconocimiento – NoComercial – CompartirIgual (by-nc-sa): No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original. NOTA: Esta es una obra derivada de la obra original realizada por Carlos Cacho y Raquel Torres.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:



Importante



Atención



Interesante

ÍNDICE DE CONTENIDO

1. Introducción	4
2. Propiedades	4
3. Arrays unidimensionales (vectores)	5
3.1 Declaración	5
3.2 Instancia	5
3.3 Almacenamiento	6
3.4 Longitud de un vector.....	6
3.5 Recorrido de un vector.....	7
3.6 La excepción <code>ArrayIndexOutOfBoundsException</code>	7
3.7 Copia de vectores	8
3.8 Cambiar el tamaño de un vector	10
4. Arrays multidimensionales	11
4.1 Arrays bidimensionales (matrices).....	11
4.2 Arrays tridimensionales (cubos u ortoedros)	12
5. La clase <code>Arrays</code>.....	14
6. La clase <code>String</code>.....	17
6.1 Comparación de strings	17
6.2 Métodos más utilizados.....	18
6.3 Lectura con <code>Scanner</code>	22
6.4 Conversiones entre <code>String</code> y otros tipos	22
7. Expresiones regulares	24
8. Búsqueda en vectores	27
8.1 Búsqueda secuencial	27
8.2 Búsqueda dicotómica o binaria.....	27
9. Ordenación de vectores	28
10. Ejemplo de llenado y recorrido de un vector	29

UD4. ESTRUCTURAS DE DATOS ESTÁTICAS

1. INTRODUCCIÓN



Un **array** o **vector** es un conjunto de valores de un **mismo tipo** dentro de una misma variable, de forma que se puede acceder a cada valor independientemente.

Para Java, además, un array es un objeto que tiene propiedades que se pueden manipular. Los arrays solucionan problemas concernientes al manejo de muchas variables que se refieren a datos similares. Por ejemplo, si tuviéramos la necesidad de almacenar las notas de una clase con 18 alumnos, necesitaríamos 18 variables diferentes, con la tremenda lentitud de manejo que supone eso. Solamente calcular la nota media requeriría una tremenda línea de código. Almacenar las notas supondría al menos 18 líneas de código.

En lugar de crear 18 variables, sería mucho mejor crear un array de tamaño 18 (es como si tuviéramos una sola variable que puede almacenar varios valores).

Gracias a los arrays se puede crear un conjunto de variables con el mismo nombre. La diferencia será que un número (índice del array) distinguirá a cada variable.

2. PROPIEDADES

Algunas propiedades de los arrays son:

- Los arrays se utilizan como contenedores para almacenar datos relacionados (en lugar de declarar variables por separado para cada uno de los elementos del array).
- **Todos los datos incluidos en el array son del mismo tipo.** Se pueden crear arrays de enteros (de tipo *int*, *long*...) o de números reales (de tipo *float*, *double*), pero **en un mismo array no se pueden mezclar tipos de datos**, por ej. *int* y *float* a la vez.
- El tamaño del array se establece cuando se crea el array (con el operador *new*, igual que cualquier otro objeto) y no se puede cambiar posteriormente, por eso los arrays son estructuras **estáticas**. Si deseáramos redimensionar la estructura en tiempo de ejecución tendríamos que emplear las “Colecciones”, que son estructuras dinámicas.
- A los elementos del array se accederá a través de la posición que ocupan dentro del conjunto de elementos del array.
- Los arrays unidimensionales se conocen con el nombre de **vectores**.
- Los arrays bidimensionales se conocen con el nombre de **matrices**.

3. ARRAYS UNIDIMENSIONALES (VECTORES)

3.1 Declaración

Un array se declara de forma similar a una variable simple pero añadiendo corchetes para indicar que se trata de un array y no de una variable simple del tipo especificado.

```
int[] numeros = new int[10];           // Array de 10 enteros

float[] numeros;
int n;
System.out.println("Escribe el tamaño del array:");
n = sc.nextInt();
numeros = new float[n];                // Array de n floats
```

Los corchetes también se pueden colocar después del nombre de la variable, aunque recomendamos colocarlos antes, así podremos ver fácilmente que esta variable es un array:

```
int numeros[] = ...
```

Ejemplos:

```
int[] notas;
double[] cuentas;
```

Declara un array de tipo int y otro de tipo double. Esta declaración indica para qué servirá el array, pero **no reserva espacio en la memoria RAM al no saberse todavía el tamaño del mismo**. Todavía no puede utilizarse el array, falta instanciarlo mediante el uso de “new” (ver ejemplo superior).

3.2 Instancia

Tras la declaración del array, se tiene que instanciar (reservar memoria), para ello se utiliza el operador **new**, que es el que realmente crea el array indicando un tamaño. Cuando se usa **new** es cuando se reserva el espacio necesario en memoria. Un array no inicializado es un array **null** (nulo o sin valor).

Ejemplo:

```
int notas[]; // Declaramos 'notas' como array de tipo int
notas = new int[3]; // Instanciamos 'notas' a tamaño 3 (creamos 3 huecos)

// Es habitual declarar el contenedor e instanciar en una sola línea
int notas[] = new int[3];
```

En el ejemplo anterior se crea un array de 3 enteros (con los tipos básicos se crea en memoria el

array y se inicializan los valores, los números se inicializan a 0).

3.3 Almacenamiento

Los valores del array se asignan (almacenan) de la misma forma que asignamos valores a una variable simple, pero en este caso utilizando, además, el índice de la posición a ocupar corchetes.

✈ El primer elemento del vector siempre estará en la posición o índice 0

Índices →	0	1	2	3	4
Vector →	8	10	2	3	5

Por ejemplo, para almacenar el valor 8 en la **tercera** posición del array escribiríamos:

```
notas[2] = 8; //Ojo, recuerda que la tercera posición es el índice 2
```

También se pueden asignar valores al array en la propia declaración e instancia:

```
int[] notas = {8, 10, 2, 3, 5};  
int[] notas2 = new int[] {8, 10, 2, 3, 5}; //Equivalente a la anterior
```

El ejemplo anterior declara e inicializa un array de 5 elementos. El ejemplo sería equivalente a:

```
int[] notas = new int[5];  
notas[0] = 8;  
notas[1] = 10;  
notas[2] = 2;  
notas[3] = 3;  
notas[4] = 5;
```

En Java (como en otros lenguajes) el primer elemento de un array está en la posición **cero**. El primer elemento del array **notas**, es **notas[0]**.

📖 Se pueden declarar arrays de cualquier tipo de datos (enteros, booleanos, doubles, ... e incluso de objetos).

3.4 Longitud de un vector

Los arrays poseen una propiedad llamada **length** que indica su tamaño.

Ejemplo:

```
int[] notas = new int[4]; // Declara e instancia vector int de tamaño 4  
System.out.println( notas.length ); // Mostrará un 4 (es su tamaño)
```

Si el vector tiene como en el ejemplo 4 elementos, la propiedad *length* nos devolverá el valor entero 4, pero su primer elemento se encuentra en `notas[0]` y el último en `notas[3]`.

3.5 Recorrido de un vector

Para recorrer un vector (acceder a todos sus elementos) siempre será necesario un **bucle**. En el siguiente ejemplo declaramos e instanciamos un vector de tipo `int` con las notas de un alumno y luego utilizamos un bucle `for` para recorrer el vector y mostrar todos los elementos.

```
// Declaramos e instanciamos vector tipo int
int[] notas = new int[] {7, 3, 9, 6, 5};

// Como es de tamaño 5 sus elementos estarán en las posiciones 0 a 4
// Recorremos el vector desde i=0 mientras i<5 (desde 0 hasta 4)
for (int i = 0; i < notas.length; i++) {
    System.out.println(notas[i]);
}
```

¿Y si quisiéramos recorrerlo al revés (del final hacia el principio)?, ¿qué cambiarías en el bucle `for`?

Ahora vamos a calcular la nota media (sumar todas y luego dividir entre el número de notas):

```
// Declaramos suma y media
int suma = 0;
float media;

// Recorremos el vector de 0 a 4, acumulando las notas en "suma"
for (int i = 0; i < notas.length; i++) {
    suma += notas[i];    // Equivale a: suma = suma + notas[i]
}

// Calculamos la media y la mostramos por pantalla
media = (float)suma / notas.length;
System.out.println("La nota media es: " + media);
```

3.6 La excepción `ArrayIndexOutOfBoundsException`

Cuando accedemos a un array cualquiera, podemos cometer el error de hacerlo en una posición inexistente. En ese caso se generará una excepción (error): **`ArrayIndexOutOfBoundsException`**. Recuerda que todo array se indexa (se referencia) desde el valor 0 hasta el valor `length-1`. Veamos ahora un ejemplo diferente de cómo se produciría este error:

```
String[] personas = new String[] { "Carolina", "Andrés" };

// Se acaba de crear un array de Strings con este contenido:
// personas[0]: "Carolina"
// personas[1]: "Andrés"
// Atención: no existe el índice 2, sólo el 0 y el 1

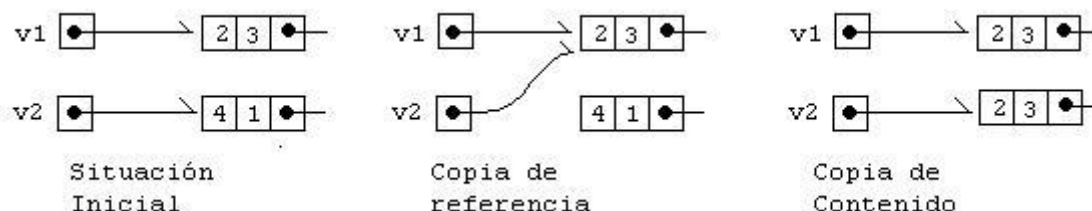
System.out.println(personas[2]); // lanzará el error
```

Salida en pantalla:

Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: [...]

3.7 Copia de vectores

Para copiar vectores no basta con igualar un vector a otro como si fuera una variable simple. Si partimos de dos vectores v1, v2 e hiciéramos v2 = v1, lo que ocurriría sería que v2 apuntaría a la posición de memoria de v1. Eso es lo que se denomina una copia de referencia:



¿Por qué ocurre esto?

El aspecto más importante de los arrays es que son **objetos** y no tipos primitivos. Toda variable que no es de un tipo primitivo es una variable de **referencia** (concepto relacionado con los punteros en otros lenguajes de programación). Solo hay 8 tipos primitivos (byte, short, int, long, float, double, boolean, char). Todo los demás son **objetos**.

Las variables de referencia no almacenan al objeto (valor) sino la referencia (dirección de memoria) de ese objeto. Si, por ejemplo, queremos copiar todos los elementos del vector v2 en el vector v1, existen dos formas para hacerlo:

1. Copiar los elementos uno a uno “manualmente”

```
for (int i = 0; i < v1.length; i++)
    v2[i] = v1[i];
// Ojo, los tamaños han de ser iguales, sino...
```

2. Utilizar la función arraycopy

```
System.arraycopy(v_origen, i_origen, v_destino, i_destino, length);
```

- ✓ *v_origen: Vector origen*
- ✓ *i_origen: Posición inicial de la copia*
- ✓ *v_destino: Vector destino*
- ✓ *i_destino: Posición inicial de la copia en destino*
- ✓ *length: Cantidad de elementos a copiar*

```
// Copiamos todos los elementos de v1 en v2
System.arraycopy(v1, 0, v2, 0, v1.length);
```


Otro ejemplo:

```
//Con variables primitivas
int num1 = 7;
int num2;

num2 = num1;
num2 = 4;
//En este punto num1 -> 7 y num2 -> 4
-----
//Con variables de referencia: creo 2 variables, pero solo un array
int[] enteros1 = new int[4];
int[] enteros2;

enteros2 = enteros1;
// Ambas variables "tienen" (señalan) a la misma información.
// Y es la referencia (dirección) a un mismo array. Por ej: @F456AC

enteros1[0] = 6;
enteros2[0] = 9;

/*
- Tanto enteros1[0] como enteros2[0] guardan el valor int 9
- System.out.println(enteros1) -> @F456AC y System.out.println(enteros2) ->
@F456AC
- Ambas variables referencian al mismo array y desde ambas variables accedo
al mismo array
*/
```

3.8 Cambiar el tamaño de un vector

¿Es posible hacer esto? La respuesta simple es **no**; una vez que se ha creado un array no se puede cambiar su tamaño. Sin embargo, lo más parecido que podemos hacer es crear un nuevo array con el tamaño apropiado y copiar los elementos del array existente al nuevo array (cuidado con los tamaños).

Veamos un ejemplo. Supongamos que partimos de este array y sus datos:

```
String[] ciudades = new String[3]; // array creado con tamaño 3.  
ciudades[0] = "Nueva York";  
ciudades[1] = "Londrés";  
ciudades[2] = "Berlin";
```

Tras eso, en nuestro programa, surge la necesidad de almacenar otra ciudad: "Sidney", pero vemos que no cabe. ¿Cómo podemos resolver la situación? Así:

1. Crearemos un nuevo array con un tamaño superior al anterior (length+1)
2. Copiaremos los 3 elementos del vector anterior al nuevo.
3. Almacenaremos el nuevo valor (Sidney) al final de nuestro nuevo vector.

```
String[] newArray = new String[ciudades.length + 1]; // Paso 1  
System.arraycopy(ciudades, 0, newArray, 0, ciudades.length); // Paso 2  
newArray[ciudades.length] = "Sydney"; // Paso 3
```

Sin embargo, hay que tener claro que esta operación no es eficiente. En Java, como en muchos otros lenguajes, existen otras herramientas más apropiadas para almacenar la información y redimensionar sin tener que hacer lo anterior. Estas herramientas son las estructuras dinámicas (en Java llamadas Colecciones) que veremos en una unidad posterior.

Tendríamos un problema similar si quisiéramos "eliminar" un elemento de un array. Por tanto, a estas alturas ya debe quedar claro que los arrays no son estructuras adecuadas para añadir y eliminar elementos en tiempo de ejecución.

4. ARRAYS MULTIDIMENSIONALES

En programación, en muchas ocasiones se necesita almacenar y gestionar datos en estructuras más complejas que un vector. Los arrays multidimensionales son una herramienta poderosa que nos permite representar datos en múltiples dimensiones, creando una matriz de valores organizados de manera ordenada. Mientras que los unidimensionales son adecuados para una lista de elementos, los multidimensionales permiten organizar datos en forma de tablas, cubos o incluso estructuras más complejas.

4.1 Arrays bidimensionales (matrices)

Los arrays multidimensionales más utilizados son los **de 2 dimensiones**, conocidos como **matrices**.



Las matrices se definen de las siguientes formas:

```
tipo identificador[][];  
tipo[][] identificador; //Mejor este que el anterior
```

Veamos un ejemplo en el que declaramos e instanciamos un array de 3 filas x 6 columnas para almacenar las notas de 3 alumnos (la fila corresponde a un alumno, y cada columna a las notas de dicho alumno):

```
int notas[][] = new int[3][6];  
// Sería como tener 3 vectores de tamaño 6
```

Accedemos a sus valores utilizando dobles corchetes.

```
precios[0][0] = 7.5;  
precios[0][1] = 12;  
precios[0][2] = 0.99;  
precios[1][0] = 4.75;  
// etc.
```

	Columnas		
	0	1	2
0	(0,0)	(0,1)	(0,2)
1	(1,0)	(1,1)	(1,2)
2	(2,0)	(2,1)	(2,2)

Suponiendo que las notas ya están almacenadas, vamos a recorrer la matriz (array bidimensional) para mostrar las notas por pantalla. Hay que tener en cuenta que como tiene dos dimensiones necesitaremos un bucle anidado (uno para las filas y otro para las columnas de cada fila):

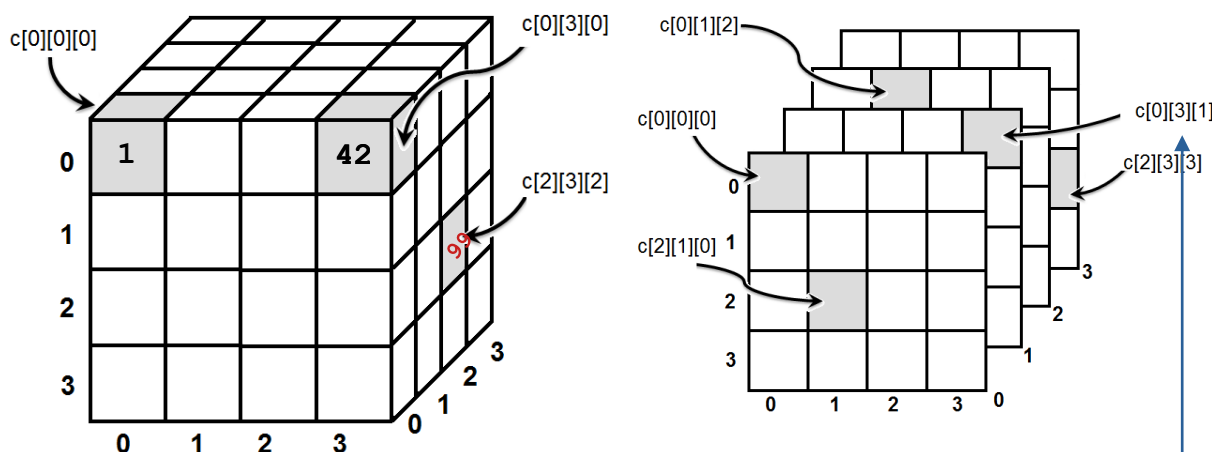
```
// Para cada fila (alumno)  
for (int i = 0; i < notas.length; i++) {  
    System.out.print("Notas del alumno " + i + ": ");  
    // Para cada columna (nota)  
    for (int j = 0; j < notas[i].length; j++) {  
        System.out.print(notas[i][j] + " ");  
    }  
}
```

Fíjate en: `notas[i].length`

4.2 Arrays tridimensionales (cubos u ortoedros)

Los arrays tridimensionales, llamados "cubos" (ortoedros si son irregulares), añaden una tercera dimensión a la estructura de datos basándose en la estructura anterior (matriz). Esto es útil cuando se necesita representar datos en un espacio tridimensional, como una colección de datos volumétricos o una matriz cúbica, o varias tablas que van unidas.

Veamos una representación en 2 figuras equivalentes (array llamado "c"):



Fijémonos en que el cubo, realmente, es como tener varias matrices unidas (varias tablas).

En Java, la declaración y manipulación de arrays tridimensionales se hace añadiendo una dimensión más a la matriz. Veamos un ejemplo completo:

```
// Declaración e inicialización de un array tridimensional de enteros
int[][][] cubo = new int[3][3][3];

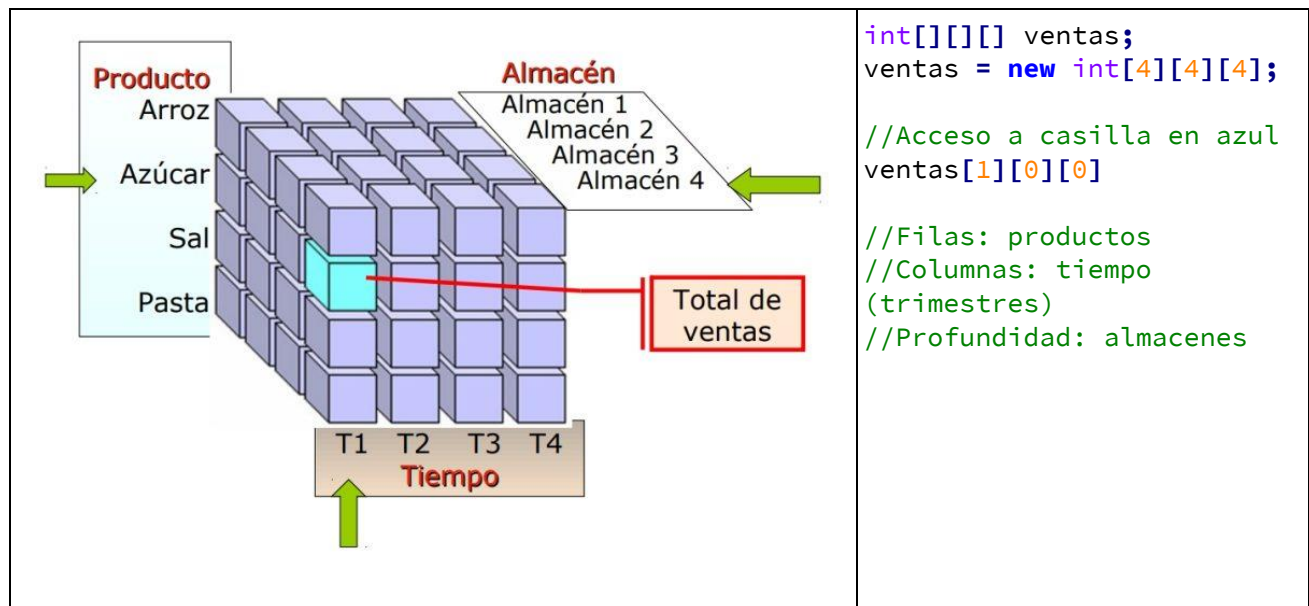
// Asignación de valores a elementos individuales del cubo
cubo[0][0][0] = 1;
cubo[0][3][0] = 42;
cubo[2][3][2] = 99;

// Acceso a elementos del cubo
// Elemento en la 1ª fila, 4ª columna y 2ª profundidad
int valor = cubo[0][3][1];

// Recorriendo un array tridimensional con bucles
for (int i = 0; i < cubo.length; i++) {
    for (int j = 0; j < cubo[i].length; j++) {
        for (int k = 0; k < cubo[i][j].length; k++) {
            System.out.println("cubo["+i+"]["+j+"]["+k+"] = " + cubo[i][j][k]);
        }
    }
}
```

Los cubos son estructuras muy útiles para almacenar información como la que vemos a

continuación. Este cubo almacena las ventas por producto, almacén y tiempo (trimestre).



5. LA CLASE ARRAYS

En el paquete *java.util* se encuentra una clase estática llamada **Arrays**. Esta clase estática permite ser utilizada como si fuera un objeto (como ocurre con *Math*) y posee **métodos** muy interesantes para utilizar sobre arrays.

Su uso es:

```
Arrays.método(argumentos);
```

La clase *Arrays* es una clase con métodos *static* útiles para trabajar con arrays. Como sus métodos son estáticos, no necesitamos crear un objeto de esa clase (instanciar la clase), sino que al igual que con la clase *Math*, llamaremos a cada método precedido por el nombre de la clase.

```
int[] arrayEnteros = { 4, 7, -45, 456, 0};
Arrays.sort(arrayEnteros);
//El array queda ordenado → {-45, 0, 4, 7, 456}
```

Métodos más destacados de la Clase Arrays

Método	Descripción	Parámetros que recibe	Tipo de datos que devuelve
boolean equals (array1, array2)	Indica si el contenido de ambos arrays es el mismo	2 arrays del mismo tipo	true si son iguales, false en caso contrario
String toString (array)	Devuelve un String con la representación del contenido del array	Un array unidimensional de cualquier tipo	Un String con la representación
void fill (array, valor)	Rellena el array con el valor indicado	Un array de cualquier dimensión y un valor del mismo tipo	Nada
tipo[] copyOf (array);	Devuelve otro array que es una copia del indicado	Un array unidimensional	1 array del mismo tipo que contiene lo mismo que el indicado
tipo[] copyOfRange (array, inicio, fin)	Devuelve otro array que es una copia del indicado desde el índice inicio hasta el índice fin	Un array unidimensional y dos enteros indicando el rango	1 array del mismo tipo que contiene una copia del rango indicado
int binarySearch (array, valor)	Busca un valor dentro de un array del tipo de valor (El array debe estar ordenado)	Un array y un valor del mismo tipo de datos	int con la posición en la que lo encuentra, o -1 si no.
void sort (array)	Ordena el array. Los tipos que contiene deben ser ordenables (primitivos, u objetos con <i>compareTo()</i>)	Un array unidimensional de cualquier tipo	Nada

Detalle de algunos métodos:

- **fill**: permite rellenar todo un array unidimensional con un determinado valor. Sus argumentos son el array a rellenar y el valor deseado:

Por ejemplo, llenar un array de 23 elementos enteros con el valor -1

```
int valores[] = new int[23];  
Arrays.fill(valores,-1); // Almacena -1 en todo el array
```

También permite decidir desde qué índice hasta qué índice rellenamos:

```
Arrays.fill(valores,5,8,-1);  
// Almacena -1 desde el 5º a la 7º posición
```

Otro ejemplo, rellenar un array de Strings:

```
String[] array2 = new String[3];  
Arrays.fill(array2, "abc");  
//Resultado: { "abc", "abc", "abc" }
```

- **equals**: Compara dos arrays y devuelve true si son iguales (false en caso contrario). Se consideran iguales si son del mismo tipo, tamaño y contienen los mismos valores.

```
Arrays.equals(valoresA, valoresB);  
// devuelve true si ambos arrays son iguales
```

- **sort**: Permite ordenar un array en orden ascendente. Se pueden ordenar sólo una serie de elementos desde un determinado punto hasta un determinado punto.

```
int[] x = {4,5,2,3,7,8,2,3,9,5};  
Arrays.sort(x); // Ordena x de menor a mayor  
Arrays.sort(x,2,5); // Ordena x solo desde 2º al 4º elemento
```

- **binarySearch**: Permite buscar un elemento de forma *ultrarrápida* en un array **ordenado**. Devuelve el índice en el que está colocado el elemento buscado. Ejemplo:

```
int[] x = {1,2,3,4,5,6,7,8,9,10,11,12};  
Arrays.sort(x);  
System.out.println(Arrays.binarySearch(x,8)); //Devolvería 7
```

Más ejemplos

```
//Compruebo si 2 arrays contienen lo mismo
if(Arrays.equals(enteros1, enteros2)){
    System.out.println("Contienen los mismos valores");
}

//Copio el array enteros1 y guardo la referencia en la variable copiaEnteros
int[] copiaEnteros = Arrays.copyOf(enteros1);

//Relleno la matriz con el caracter '*'
char[][] caracteres = new char[50][100];
Arrays.fill(caracteres, '*');

//Muestro por pantalla una representacion del array
int[] numeros = {-4, 6, 0, -34};
System.out.println(Arrays.toString(numeros));
```


6. LA CLASE STRING

Las cadenas de texto son objetos especiales. Parece que tienen un funcionamiento similar a los tipos primitivos, pero a diferencia de estos, tienen una serie de métodos. Además, es un tipo de objeto con algunos privilegios. Un objeto **String** representa una cadena de caracteres, y puede tener 0 ó más caracteres. Su representación **literal** se rodea con comillas dobles: **"Hola 123 Paco"**.

Los textos deben manejarse instanciando objetos de tipo **String** (como inicializar variables).

Ejemplo:

```
String texto1 = "¡Prueba de texto!";
```

Las cadenas pueden ocupar varias líneas utilizando el operador de concatenación "+".

```
String texto2 = "Este es un texto que ocupa " +  
                "varias líneas, no obstante, se puede " +  
                "perfectamente encadenar";
```

También se pueden crear objetos *String* sin utilizar constantes entrecomilladas, usando otros constructores:

```
char[] palabra = {'P', 'a', 'l', 'a', 'b', 'r', 'a'}; // Array de char  
String cadena = new String(palabra);
```

6.1 Comparación de strings

⚡ Los objetos *String* NO pueden compararse directamente con los operadores de comparación `==` como con los tipos de datos simples, dado que son **objetos**.

En su lugar se deben utilizar estos métodos:

- **cadena1.equals(cadena2)**. El resultado es true si la cadena1 es igual a la cadena2. Ambas cadenas son variables de tipo *String*.
- **cadena1.equalsIgnoreCase(cadena2)**. Como la anterior, pero en este caso no se tienen en cuenta mayúsculas y minúsculas.
- **s1.compareTo(s2)**. Compara ambas cadenas, considerando el orden alfabético. Si la primera cadena es mayor en orden alfabético que la segunda, devuelve la diferencia positiva entre una cadena y otra, si son iguales devuelve 0 y si es la segunda la mayor, devuelve la diferencia negativa entre una cadena y otra. Hay que tener en cuenta que el orden no es el del alfabeto español, sino que usa la tabla ASCII, en esa tabla la letra ñ es mucho mayor que la o.
- **s1.compareToIgnoreCase(s2)**. Igual que la anterior, sólo que además ignora las mayúsculas.

6.2 Métodos más utilizados

Son funciones que poseen los propios objetos de tipo *String*. Para utilizarlos basta con poner el nombre del método y sus parámetros después del nombre del objeto *String*.

```
objetoString.metodo(parámetros);
```

A continuación, veamos algunos de los métodos de la clase *String* más usados:

Nombre	Función	Parámetros que recibe	Valor que devuelve
<code>length()</code>	Indica la longitud en caracteres del <i>String</i>	No recibe nada	Devuelve un <i>int</i> con la cantidad de caracteres
<code>charAt()</code>	Obtiene el caracter (<i>char</i>) en la posición indicada	Un <i>int</i> indicando la posición (Empieza en 0)	El <i>char</i> de esa posición
<code>equals()</code>	Indica si dos <i>Strings</i> son iguales	Recibe otro <i>String</i>	true si son iguales
<code>equalsIgnoreCase()</code>	Indica si dos <i>Strings</i> son iguales sin atender a mayúsculas-minúsculas	otro <i>String</i>	true si son iguales
<code>contains()</code>	Indica si una cadena contiene a otra pasada por parámetro	otro <i>String</i>	true si la contiene, false si no
<code>compareTo()</code>	Comprueba si una cadena es mayor, igual o menor a otra, alfabéticamente (tabla ascii)	otro <i>String</i>	Un <i>int</i> <0, 0, ó >0 si el parámetro es menor, igual o mayor a la cadena.
<code>substring()</code>	Obtiene una subcadena con el rango de caracteres desde inicio a fin	Dos <i>ints</i> , <u>inicio</u> y <u>fin</u> , indicando el rango de la subcadena	una subcadena <i>String</i> con el rango de caracteres desde inicio hasta fin
<code>substring()</code>	Obtiene una subcadena <i>String</i> desde el valor inicio, hasta el final de la cadena	un solo <i>int</i> de inicio	subcadena <i>String</i> desde el caracter indicando en inicio, hasta el final de la cadena
<code>valueOf()</code>	Convierte el valor recibido a un <i>String</i>	Un elemento de cualquier tipo primitivo	<i>String</i> con la representación
<code>replaceAll()</code>	Reemplaza la aparición de una serie de caracteres por otros	Dos <i>Strings</i> con los caracteres a reemplazar, y el valor del reemplazo	<i>String</i> con el resultado del reemplazo
<code>indexOf()</code>	Busca el valor recibido a partir de una posición dada e indica la posición donde lo encuentra	(2 parámetros) <i>String</i> ó un <i>char</i> con el valor y un entero con la posición de inicio de la búsqueda	<i>int</i> con la posición en la que lo encuentra ó -1 si no lo encuentra
<code>startsWith()</code>	Indica si una cadena empieza por otra cadena recibida por parámetro	un <i>String</i>	true si la cadena comienza por la cadena recibida por parámetro
<code>lastIndexOf()</code>	Devuelve la posición de la última aparición de un valor	<i>String</i> ó <i>char</i> con el valor a buscar	Un <i>int</i> con la posición, ó -1 si no se encuentra

Nombre	Función	Parámetros que recibe	Valor que devuelve
<code>endsWith()</code>	Indica si una cadena termina por otra cadena recibida por parámetro	un <code>String</code>	<code>true</code> si la cadena termina por la cadena recibida por parámetro
<code>toLowerCase()</code>	Convierte el <code>String</code> a minúsculas	Nada	El <code>String</code> en minúsculas
<code>toUpperCase()</code>	Convierte el <code>String</code> a mayúsculas	Nada	El <code>String</code> en mayúsculas
<code>split()</code>	Crea un <i>array</i> resultado de dividir el <code>String</code> por el valor separador	<i>String</i> con el valor separador	Un array con tantas celdas, como divisiones tenga el <i>String</i> por el valor separador indicado
<code>toCharArray()</code>	Obtiene un array de caracteres	Nada	Un array con los caracteres del <code>String</code>
<code>toString(array)</code>	Devuelve un <code>String</code> con la representación del contenido de un array	Un array unidimensional de cualquier tipo	Un <code>String</code> con la representación

Algunos ejemplos más detallados:

valueOf: Convierte valores que no son de cadena a forma de cadena.

```
String numero = String.valueOf(1234);
// Convierte el número int 1234 en el String "1234"
```

length: Devuelve la longitud de una cadena (el número de caracteres de la cadena):

```
String texto1 = "Prueba";
System.out.println(texto1.length()); // Escribe un 6
```

Concatenar cadenas: se puede hacer de 2 formas, utilizando el método *concat* o con el operador *+*.

```
String s1 = "Buenos ", s2 = " días", s3, s4;
s3 = s1 + s2;
s4 = s1.concat(s2);
```

En ambos casos (*s3* y *s4*), el contenido de *s3* y *s4* sería el mismo: "Buenos días".

charAt: Devuelve un carácter concreto de la cadena. El carácter a devolver se indica por su posición (el primer carácter es la posición 0). Si la posición es negativa o sobrepasa el tamaño de la cadena, ocurre un error de ejecución, una excepción tipo *IndexOutOfBoundsException-Exception* (recuerda este tipo de error, se repetirá muchas veces).

```
String s1 = "Prueba";
char c1 = s1.charAt(2); // c1 valdrá 'u'
```

substring : Da como resultado una porción del texto de la cadena. La porción se toma desde una posición inicial hasta una posición final (sin incluir esa posición final). Si las posiciones indicadas no son válidas ocurre una excepción de tipo ***IndexOutOfBoundsException***. Se empieza a contar desde la posición cero.

```
String s1 = "Buenos días";  
String s2 = s1.substring(7,10); // s2 = "día"
```

indexOf : Devuelve la primera posición en la que aparece un determinado texto en la cadena. En el caso de que la cadena buscada no se encuentre, devuelve -1. El texto a buscar puede ser *char* o *String*.

```
String s1 = "Quería decirte que quiero que te vayas";  
System.out.println(s1.indexOf("que")); // Devuelve 15
```

También se puede buscar desde una determinada posición:

```
String s1 = "Quería decirte que quiero que te vayas";  
System.out.println(s1.indexOf("que",16)); // Ahora devolvería 26
```

lastIndexOf : Devuelve la última posición en la que aparece un determinado texto en la cadena. Es casi idéntica a la anterior, sólo que busca desde el final.

```
String s1 = "Quería decirte que quiero que te vayas";  
System.out.println(s1.lastIndexOf("que")); //Devolvería 26
```

También permite comenzar a buscar desde una determinada posición.

endsWith : Devuelve true si la cadena termina con un determinado texto.

```
String s1 = "Quería decirte que quiero que te vayas";  
System.out.println(s1.endsWith("vayas")); //Devolvería true
```

startsWith : Devuelve true si la cadena empieza con un determinado texto.

```
String s1 = "Quería decirte que quiero que te vayas";  
System.out.println(s1.startsWith("vayas")); // Devolvería false
```

replace: Cambia todas las apariciones de un carácter (o caracteres) por otro/s en el texto que se indique y lo almacena como resultado. El texto original no se cambia, por lo que hay que asignar el resultado de replace a un *String* para almacenar el texto cambiado.

Ejemplo1

```
String s1 = "Mariposa";  
System.out.println(s1.replace('a', 'e')); //Devuelve "Meripose"  
System.out.println(s1); //Sigue valiendo "Mariposa"
```

Para guardar el valor deberíamos hacer:

```
String s2 = s1.replace('a', 'e');
```

Ejemplo2

```
String s1 = "Buscar armadillos";  
System.out.println(s1.replace("ar", "er"));  
//Devuelve "Buscar ermadillos"  
System.out.println(s1); //Sigue valiendo "Buscar armadillos"
```

toUpperCase: Obtiene la versión en mayúsculas de la cadena. Es capaz de transformar todos los caracteres nacionales:

```
String s1 = "Batallón de cigüeñas";  
System.out.println(s1.toUpperCase());  
//Escribe: BATALLÓN DE CIGÜEÑAS (pero NO modifica a s1)
```

toLowerCase: Obtiene la versión en minúsculas de la cadena.

toCharArray: Consigue un array de caracteres a partir de una cadena. De esa forma podemos utilizar las características de los arrays para manipular el texto, lo cual puede ser interesante para manipulaciones complicadas.

```
String s="texto de prueba";  
char c[]=s.toCharArray();
```

format: Modifica el formato de la cadena a mostrar. Muy útil para mostrar sólo los decimales que necesitamos de un número decimal. Indicaremos "%" para indicar la parte entera más el número de decimales a mostrar seguido de una "f":

```
System.out.println(String.format("%.2f", number));  
// Muestra el número con dos decimales.
```

split: Consigue un array de strings a partir de un string basándose en un separador especificado por el usuario:

```
String s1 = new String("C++#Java#Python#Eiffel#");  
String[] lenguajesP00 = s1.split("#");
```

6.3 Lectura con Scanner

Como ya sabemos, la lectura de un *String* utilizando la clase *Scanner* se realiza con el método *nextLine()*:

```
Scanner in = new Scanner(System.in);
String s = in.nextLine();
```

Recuerda que, si leemos un tipo de dato numérico antes de leer un *String* deberemos limpiar el buffer de entrada, de lo contrario leerá el valor '\n' (salto de línea) introducido después del número y se lo asignará a la variable *String*, con lo que no se leerá bien la entrada. También se pueden leer todos los datos como *String* y “parsearlo” al valor correcto.

6.4 Conversiones entre String y otros tipos

Operador de concatenación

El operador + permite unir cadenas:

```
String cadena1 = "Mi";
String cadena2 = " primer";
String cadena3 = cadena1 + cadena2 + " programa";

System.out.println(cadena3); -> "Mi primer programa"
```

Siempre que concatenemos distintos tipos de datos con un *String*, da como resultado otro *String*:

```
int numero = 99;
double decimal = -35.14;

String cadena = numero + decimal + "";
//El resultado no es una suma, sino una concatenación: "99-35.14"
```

Conversiones

No se puede hacer un casting directo entre tipos de datos que no tienen nada que ver (cadenas de caracteres y tipos primitivos). Debemos utilizar métodos que nos ayuden a hacer esas conversiones.

Convertir de String a tipo primitivo: para poder convertir desde un valor contenido en un *String* al tipo primitivo de ese valor, usaremos los métodos de una serie de clases:

```
String cadenaEntero = "123";
int entero = Integer.parseInt(cadenaEntero);
boolean valor = Boolean.parseBoolean("true");
double decimal = Double.parseDouble("23.45");
```

Estas clases son conocidas como **clases envoltorio**, ya que se usan para envolver a tipos primitivos. Existe una clase por cada uno de los 8 tipos primitivos de datos.

Convertir de tipo primitivo a String: para convertir de un tipo primitivo a String tenemos dos formas:

```
int numero = 46;
String cadenaNumerica = String.valueOf(numero);

double decimal = 34.75;
String cadenaDecimal = String.valueOf(decimal);

//El resultado de concatenar cualquier valor con un String, es otro String
String cadenaNumerica = numero + "";
```

7. EXPRESIONES REGULARES

Una expresión regular es un patrón que describe una cadena de caracteres cualquiera. Por ejemplo, cuando usamos la expresión *.doc para buscar archivos en la terminal del sistema, estamos empleando un patrón (el asterisco) que representa a todos los archivos que terminen en extensión doc, en cierto modo esto sería una expresión regular muy sencilla.

Una expresión regular nos servirá para buscar patrones en una cadena de texto, por ejemplo, comprobar que una fecha tiene la estructura correcta (dos dígitos, luego un punto, después otros dos dígitos para el mes, otro punto y cuatro dígitos para el año), también para comprobar que un email está bien escrito (letra y números, luego una arroba, más texto, un punto final y otro texto más), etc. Para cada uno de estos casos existe una expresión regular que los representa. Hay que destacar que la expresión regular solo va a comprobar la sintaxis, es decir el formato, pero no va a validar la lógica semántica, por ejemplo, en una fecha no va a verificar si un 31 de abril es correcto o no.

Las expresiones regulares se aplican a distintas tareas, aunque la más habitual será comprobar si una cadena cumple con un determinado patrón representado por la expresión regular. En Java, podemos utilizar diferentes estrategias para afrontar las expresiones regulares. Podemos usar un método de la clase String, o podemos usar clases más avanzadas como Matcher y Pattern del paquete java.util.regex.

Veamos con un ejemplo las 3 formas de comprobar si una cadena cumple una expresión regular. En este caso comprobaríamos si la cadena llamada "txt" cumple con el formato especificado por el patrón (o expresión regular) almacenado en "regex":

```
boolean resultado1, resultado2, resultado3;
String txt = "12345678A";
String regex = "\\d{8}[A-Z]";

//Opción 1 (más fácil, más limitada)
resultado1 = txt.matches(regex);

//Opción 2 (similar a la anterior)
resultado2 = Pattern.matches(regex, txt);

//Opción 3 (más potente, mejor si la regex se usa muchas veces)
Pattern p = Pattern.compile(regex);
Matcher m = p.matcher(txt);
resultado3 = m.matches();
```

En el ejemplo anterior estamos verificando un formato de DNI, es decir, si la cadena tiene 8 dígitos numéricos y a continuación una letra mayúscula (como debería ser un DNI). En los 3 casos devuelve 'true' si se cumple y 'false' en otro caso. Con cualquiera de las 3 formas descritas, en caso de que se cumpla la expresión, el resultado será true y si no se cumple será false, por eso está asignándose el resultado a variables boolean. La opción 3 (Pattern + Matches) es la más potente e indicada si se va a hacer un trabajo intensivo o complejo con las expresiones regulares, así como si se

van a repetir las comprobaciones de un mismo patrón muchas veces. En este curso, por simplicidad, trabajaremos sólo con el método “matches” de la clase String.

matches: este método examina la expresión regular que recibe como parámetro (en forma de *String*) y devuelve verdadero si el texto que examina cumple la expresión regular. Una expresión regular es una expresión textual que utiliza símbolos especiales para hacer búsquedas avanzadas y comprobar si una frase o texto cumple o se ajusta a un patrón definido (Ejemplo: formato del DNI, de una matrícula, de una fecha de nacimiento....).

Las expresiones regulares pueden contener:

- **Caracteres**. Como a, s, ñ,... y les interpreta tal cual. Si una expresión regular contuviera sólo un carácter, matches devolvería verdadero si el texto contiene sólo ese carácter. Si se ponen varios, obliga a que el texto tenga exactamente esos caracteres.
- **Caracteres de control**
 - \d , dígito, es igual que [0-9]
 - \D , no dígito, es igual que [^0-9]
 - \s , carácter en blanco, es igual que [\t\n\x0B\f\r]
 - \S , no carácter en blanco, es igual que [^\s]
 - \w , carácter alfanumérico, es igual que [a-zA-Z_0-9]
 - \W , no carácter alfanumérico, es igual que [^\w]
- **Opciones de caracteres**. Se ponen entre corchetes. Por ejemplo [abc] significa a, b ó c.
- **Negación de caracteres**. Funciona al revés impide que aparezcan los caracteres indicados. Se pone con corchetes dentro de los cuales se pone el carácter circunflejo (^). [^abc] significa ni a ni b ni c.
- **Rangos**. Se ponen con guiones. Por ejemplo [a-z] significa: cualquier carácter de la a a la z.
- **Intersección**. Usa &&. Por ejemplo [a-x&&r-z] significa de la r a la x (intersección de ambas expresiones).
- **Sustracción**. Ejemplo [a-x&&[^cde]] significa de la a a la x excepto la c, d ó e.
- **Cualquier carácter**. Se hace con el símbolo punto (.)
- **Opcional**. El símbolo ? sirve para indicar que la expresión que le antecede puede aparecer una o ninguna veces. Por ejemplo a? indica que puede aparecer la letra a o no.
- **Repetición**. Se usa con el asterisco (*). Indica que la expresión puede repetirse varias veces o incluso no aparecer.
- **Repetición obligada**. Lo hace el signo +. La expresión se repite una o más veces (pero al menos una).
- **Repetición un número exacto de veces**. Un número entre llaves indica las veces que se repite la expresión. Por ejemplo \d{7} significa que el texto tiene que llevar siete números (siete cifras del 0 al 9). Con una coma significa al menos, es decir \d{7,} significa al menos siete veces (podría repetirse más veces). Si aparece un segundo número indica un máximo número de veces \d{7,10} significa de siete a diez veces.

Veamos ahora algunos ejemplos de expresiones regulares y el uso de matches:

```
public static void main(String[] args) {

    String cadena = "Solo se que no se nada";

    // 1: devolverá false, ya que la cadena tiene mas caracteres
    System.out.println("ejemplo1: "+ cadena.matches("Solo"));

    // 2: devolverá true, siempre y cuando no cambiemos la cadena Solo
    System.out.println("ejemplo2: "+ cadena.matches("Solo.*"));

    // 3: devolverá true, siempre que uno de los caracteres se cumpla
    System.out.println("ejemplo3: "+ cadena.matches(".*[qnd].*"));

    // 4: devolverá false, ya que ninguno de esos caracteres estan
    System.out.println("ejemplo4: "+ cadena.matches(".*[xyz].*"));

    // 5: devolverá true, ya que le indicamos que no incluya esos caracteres
    System.out.println("ejemplo5: "+ cadena.matches(".*[^xyz].*"));

    // 6: devolverá true, si quitamos los caracteres delante de ? seguirá dando true
    System.out.println("ejemplo6: "+ cadena.matches("So?lo se qu?e no se na?da"));

    // 7: devolverá false, ya que la String empieza por una S mayúscula
    System.out.println("ejemplo7: "+ cadena.matches("[a-z].*"));

    // 8: devolverá true, a diferencia del caso anterior
    System.out.println("ejemplo8: "+ cadena.matches("[A-Z].*"));

    String cadena2 = "abc1234";

    // 9: devolverá true, ya que mínimo debe aparecer alguno de los caracteres 1 vez
    System.out.println("ejemplo9: "+ cadena2.matches("[abc]+.*"));

    // 10: devolverá true, ya que, además del ejemplo anterior, indicamos que debe
    // repetirse un valor numérico 4 veces
    System.out.println("ejemplo10: "+ cadena2.matches("[abc]+\\d{4}"));

    // 11: devolverá true, ya que, además del ejemplo anterior, indicamos que debe
    // repetirse un valor numérico entre 1 y 10 veces
    System.out.println("ejemplo11: "+ cadena2.matches("[abc]+\\d{1,10}"));

}
```

8. BÚSQUEDA EN VECTORES

Existen dos formas de buscar un elemento dentro de un vector: la búsqueda secuencial y la búsqueda dicotómica o binaria.

8.1 Búsqueda secuencial

La búsqueda secuencial es la más fácil de las dos ya que consiste en comparar los elementos del vector con el elemento a buscar (N). Un ejemplo es el siguiente, donde se devuelve la posición del elemento en el vector y si no lo encuentra, devuelve el valor -1:

```
56 public static int busquedaSecuencial(int[] v, int elemento)
57 {
58     int i, posicion = -1;
59
60     for(i = 0; i < v.length && posicion == -1; i++)
61         if(v[i] == elemento)
62             posicion = i;
63
64     return posicion;
65 }
```

¿Qué pasaría en este caso si hubiera 2 elementos iguales en distintas posiciones?

8.2 Búsqueda dicotómica o binaria

En este caso el vector debe estar ordenado. Se dividirá en dos para buscar el elemento en una parte del vector o en otra y así sucesivamente hasta encontrar, o no, el elemento. Ejemplo:

```
66 public static int busquedaDicotomica(int[] v, int elemento)
67 {
68     int izq = 0; // El índice 'izq' se establece en la posición 0
69     int der = v.length-1; // El índice 'der' se establece en la última posición
70     int centro = (izq + der)/2; // El índice 'centro' se establece en la posición
71     int posicion;
72
73     while(izq <= der && v[centro] != elemento)
74     {
75         if(elemento < v[centro])
76             der = centro - 1; // Si el elemento es menor que el centro cambiamos el índice 'der'
77         else
78             izq = centro + 1; // Sino cambiamos el índice 'izq'
79
80         centro = (izq + der)/2; // Actualizamos el centro
81     }
82
83     if(izq > der)
84         posicion = -1;
85     else
86         posicion = centro;
87
88     return posicion;
89 }
```

En este caso, al igual que en el anterior, la función devuelve la posición del elemento o -1 si no lo encuentra. Es importante ver que el vector debe estar ordenado para quedarnos con la parte desde la izquierda al centro del vector o desde el centro del vector a la derecha, dependiendo si el elemento a buscar es mayor o menor al elemento del centro del vector. Esta búsqueda es más óptima que la secuencial ya que no tiene que recorrer el vector entero.

9. ORDENACIÓN DE VECTORES

Existen diferentes algoritmos para ordenar un vector. Algunos ejemplos son:

- Burbuja
- Inserción
- Selección
- Quicksort (el más rápido de los cuatro)

Veamos, como ejemplo, el más sencillo de todos (e ineficiente), el algoritmo de la burbuja (Bubble sort) para ordenar un vector de números enteros:

```
public static void ordenarBurbuja(int[] A) {  
    int i, j, aux;  
    for (i = 0; i < A.length - 1; i++) {  
        for (j = 0; j < A.length - i - 1; j++) {  
            if (A[j + 1] < A[j]) {  
                aux = A[j + 1];  
                A[j + 1] = A[j];  
                A[j] = aux;  
            }  
        }  
    }  
}
```

Java ya tiene estos algoritmos implementados y podemos utilizar métodos de ordenación muy fácilmente (por ejemplo, con `Arrays.sort()`).

No obstante, a final de curso estudiaremos algunos ejemplos más detalladamente.

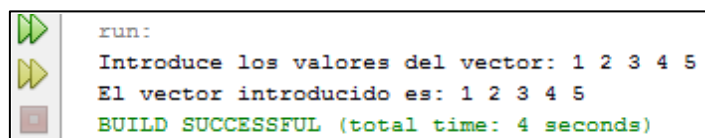
10. EJEMPLO DE LLENADO Y RECORRIDO DE UN VECTOR

Vamos a ver un ejemplo de cómo llenar y mostrar un vector de enteros:

```
19 public static void main(String[] args) {
20     Scanner in = new Scanner(System.in);
21
22     int[] vector = new int[5]; // Creación de un vector de enteros de tamaño 5
23     int i;
24
25     System.out.print("Introduce los valores del vector: ");
26
27     // Llenado del vector con valores desde teclado
28     for(i = 0; i < vector.length; i++)
29         vector[i] = in.nextInt();
30
31     System.out.print("El vector introducido es: ");
32
33     // Mostrar el vector
34     for(i = 0; i < vector.length; i++)
35         System.out.print(vector[i] + " ");
36
37     System.out.println();
38 }
39 }
```

Hacemos uso de la propiedad **length**. También podríamos haber puesto un 5 (tamaño del vector).

La salida es:



```
run:
Introduce los valores del vector: 1 2 3 4 5
El vector introducido es: 1 2 3 4 5
BUILD SUCCESSFUL (total time: 4 seconds)
```

Podemos introducir los valores con espacios y una vez introducido el quinto número darle al 'intro'.
O introducir un valor por línea.